

## Tema 4.1. Composición

---

1. En C, podemos crear estructuras mayores **componiendo** unas con otras, que suelen describirse como "A tiene-un/tiene-varios B". Pon un ejemplo, empleando **struct**, de una línea de puntos, donde puntos tienen dos coordenadas (**x** e **y**), y la línea esta hecha de dos puntos. Incluye una función para calcular la distancia entre puntos y otra para hallar la longitud de una línea.

La función sería algo así:

```
#include <math.h>

struct Punto {
    double x;
    double y;
};

struct Linea {
    struct Punto p1;
    struct Punto p2;
};

double distancia(struct Punto a, struct Punto b) {
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

double longitud(struct Linea l) {
    return distancia(l.p1, l.p2);
}
```

2. Ahora transforma ese ejemplo a orientación a objetos con Java, para tener un primer ejemplo de **composición** en orientación a objetos. Crea una clase **Punto**, y una clase **Linea**. La clase **Punto** debe tener un método para calcular distancia a otro **Punto** y **Linea** debe tener un método para calcular su longitud. Gracias a la ocultación de información, supera a C, garantizando que los puntos sean inmutables, al igual que la línea, que una vez creada, no queremos que se modifique de qué a qué puntos va dicha línea.

Sería :

```
public class Punto {
    private final double x;
```

```
private final double y;

public Punto(double x, double y) {
    this.x = x;
    this.y = y;
}

public double distancia(Punto otro) {
    double dx = this.x - otro.x;
    double dy = this.y - otro.y;
    return Math.sqrt(dx * dx + dy * dy);
}

}

public class Linea {
    private final Punto p1;
    private final Punto p2;

    public Linea(Punto p1, Punto p2) {
        this.p1 = p1;
        this.p2 = p2;
    }

    public double longitud() {
        return p1.distancia(p2);
    }
}
```

3. ¿Qué significa la **multiplicidad** en la composición? En el ejemplo anterior, ¿cuál es la multiplicidad entre **Linea** y **Punto**? Indícalo expresando la multiplicidad en ambas direcciones, de **Linea** a **Punto** y de **Punto** a **Linea**.

Sigún la multiplicidad, una clase puede tener una relación de composición con otra clase, donde una instancia de la primera clase puede contener una o varias instancias de la segunda clase. En el ejemplo anterior, la multiplicidad entre **Linea** y **Punto** es de 1 a 2, ya que cada **Linea** tiene exactamente dos **Punto**, mientras que cada **Punto** puede estar asociado a cero o más **Linea**. Por lo tanto, la multiplicidad de **Linea** a **Punto** es 1..2, y la multiplicidad de **Punto** a **Linea** es 0..\* (cero o más). Esto significa que una **Linea** siempre tiene dos **Punto**, pero un **Punto** puede no estar asociado a ninguna **Linea** o estar asociado a varias **Linea**. La multiplicidad es importante para entender la relación entre las clases y cómo se pueden instanciar los objetos en el contexto de la composición.

4. ¿Qué significa composición **fuerte** y composición **débil**? ¿Qué consecuencia implica en relación al ciclo de vida de los objetos? Indica a cuál solemos referirnos como "**asociación o agregación**" y a cuál como "**composición**" propiamente.

La composición fuerte implica que el ciclo de vida de los objetos contenidos está ligado al ciclo de vida del objeto contenedor. Es decir, si el objeto contenedor es destruido, los objetos contenidos también lo serán. En este caso, se suele referir a esta relación como "composición" propiamente dicha. Por otro lado, la

composición débil implica que el ciclo de vida de los objetos contenidos no está necesariamente ligado al ciclo de vida del objeto contenedor. En este caso, se suele referir a esta relación como "asociación" o "agregación". En una asociación o agregación, los objetos contenidos pueden existir independientemente del objeto contenedor y pueden ser compartidos entre diferentes objetos contenedores. En resumen, la composición fuerte implica una relación más estrecha entre los objetos, mientras que la composición débil permite una mayor flexibilidad en la gestión de los objetos y su ciclo de vida.

## 5. Cuando una clase usa a otra al recibirla o devolverla como parámetro en algún método, al hacer `new` dentro de un método, o al usarlas como variables locales, ¿hablamos de composición o de "**dependencia**"?

Cuando una clase usa a otra al recibirla o devolverla como parámetro en algún método, al hacer `new` dentro de un método, o al usarlas como variables locales, estamos hablando de **dependencia**. La dependencia se refiere a una relación en la que una clase utiliza a otra clase para realizar alguna función o tarea, pero no tiene una relación de propiedad o contención con esa clase. En este caso, la clase que depende de otra clase no tiene control sobre el ciclo de vida de los objetos de la clase dependida y puede existir independientemente de ella. Por lo tanto, la dependencia es una relación más débil que la composición, ya que no implica una relación de propiedad o contención entre las clases. En resumen, cuando una clase utiliza a otra clase sin tener una relación de propiedad o contención, estamos hablando de dependencia.

## 6. En el ejemplo anterior de línea y punto, programa la relación entre `Linea` y `Punto` de dos formas. Una **como composición fuerte**, donde el ciclo de vida de los puntos está ligado al de Linea y otra **como composición débil**, donde no.

### Composición fuerte:

```
public class Linea {
    private final Punto p1;
    private final Punto p2;

    public Linea(double x1, double y1, double x2, double y2) {
        this.p1 = new Punto(x1, y1);
        this.p2 = new Punto(x2, y2);
    }

    public double longitud() {
        return p1.distancia(p2);
    }
}
```

### Composición débil:

```
public class Linea {
    private Punto p1;
    private Punto p2;
```

```
public Linea(Punto p1, Punto p2) {
    this.p1 = p1;
    this.p2 = p2;
}

public double longitud() {
    return p1.distancia(p2);
}
}
```

7. En Java, en la composición fuerte, ¿cuando el contenedor destruye los objetos? No se observa que **Linea** destruya los **Punto** explícitamente, ¿Por qué?

En Java, el contenedor no destruye explícitamente los objetos contenidos en una composición fuerte. En lugar de eso, Java utiliza un sistema de recolección de basura (garbage collection) para gestionar la memoria. Cuando un objeto ya no es referenciado por ningún otro objeto, se considera elegible para la recolección de basura y eventualmente será destruido por el recolector de basura. En el caso de la composición fuerte entre **Linea** y **Punto**, cuando una instancia de **Linea** es destruida (es decir, cuando ya no hay referencias a ella), las instancias de **Punto** que contiene también se vuelven elegibles para la recolección de basura, ya que no hay referencias a ellas desde ningún otro objeto. Por lo tanto, aunque **Linea** no destruya explícitamente los **Punto**, el ciclo de vida de los **Punto** está ligado al ciclo de vida de **Linea**, y serán destruidos automáticamente por el sistema de recolección de basura cuando ya no sean necesarios.

8. Pon un ejemplo de composicion débil entre un departamento que tiene varios profesores. Implementa dos composiciones a la vez: entre el departamento y todos sus profesores y entre el departamento y su director, que es un profesor del departamento. Siempre debe haber un director en el departamento desde el inicio. Lanza excepciones si se viola la invariante. Emplea arrays primitivos de Java, estilo **Profesor[]**, con máximo 50, pero no rompas la encapsulación, no desveles que estás empleando un array, permite añadir un **Profesor** al final de la lista, y eliminar un profesor dada su posición. Da acceso a los profesores con un método para saber cuántos hay y otro para obtener un profesor por posición. El director se puede cambiar por otro profesor del departamento. Sin embargo, ten en cuenta esta invariante de clase: el director debe formar siempre parte de la lista de profesores, es decir, ten cuidado al cambiar el director o al eliminar un profesor.

```
public class Departamento {
    private Profesor director;
    private Profesor[] profesores;
    private int numProfesores;

    public Departamento(Profesor director) {
        if (director == null) {
            throw new IllegalArgumentException("El director no puede ser nulo");
        }
    }
}
```

```
    }
    this.director = director;
    this.profesores = new Profesor[50];
    this.profesores[0] = director;
    this.numProfesores = 1;
}

public void agregarProfesor(Profesor profesor) {
    if (numProfesores >= 50) {
        throw new IllegalStateException("No se pueden agregar más profesores");
    }
    profesores[numProfesores] = profesor;
    numProfesores++;
}

public void eliminarProfesor(int pos) {
    if (pos < 0 || pos >= numProfesores) {
        throw new IllegalArgumentException("Posición inválida");
    }
    if (profesores[pos].equals(director)) {
        throw new IllegalStateException("No se puede eliminar al director");
    }
    for (int i = pos; i < numProfesores - 1; i++) {
        profesores[i] = profesores[i + 1];
    }
    profesores[numProfesores - 1] = null;
    numProfesores--;
}

public int getNumProfesores() {
    return numProfesores;
}

public Profesor getProfesor(int pos) {
    if (pos < 0 || pos >= numProfesores) {
        throw new IllegalArgumentException("Posición inválida");
    }
    return profesores[pos];
}

public void cambiarDirector(Profesor nuevoDirector) {
    if (nuevoDirector == null) {
        throw new IllegalArgumentException("El nuevo director no puede ser nulo");
    }
    boolean encontrado = false;
    for (int i = 0; i < numProfesores; i++) {
        if (profesores[i].equals(nuevoDirector)) {
            encontrado = true;
            break;
        }
    }
    if (!encontrado) {
```

```

        throw new IllegalStateException("El nuevo director debe ser un
profesor del departamento");
    }
    this.director = nuevoDirector;
}
}

```

9. En Java, existen también **List**, cambia y muestra cómo sería el código anterior empleando **List** en vez de arrays primitivos. ¿Qué parte del código original te has ahorrado? Además, fíjate en el método **getProfesor(int pos)**: si en su lugar existiera un método que devolviera todos los profesores a la vez, ¿qué problema tendría devolver directamente la lista interna? ¿Cómo lo resolverías?

Al utilizar **List** en lugar de arrays primitivos, se puede simplificar el código y eliminar la necesidad de gestionar manualmente el tamaño del array y las operaciones de inserción y eliminación. El código modificado sería algo así:

```

import java.util.ArrayList;
import java.util.List;

public class Departamento {
    private Profesor director;
    private List<Profesor> profesores;

    public Departamento(Profesor director) {
        if (director == null) {
            throw new IllegalArgumentException("El director no puede ser nulo");
        }
        this.director = director;
        this.profesores = new ArrayList<>();
        this.profesores.add(director);
    }

    public void agregarProfesor(Profesor profesor) {
        if (profesores.size() >= 50) {
            throw new IllegalStateException("No se pueden agregar más
profesores");
        }
        profesores.add(profesor);
    }

    public void eliminarProfesor(int pos) {
        if (pos < 0 || pos >= profesores.size()) {
            throw new IllegalArgumentException("Posición inválida");
        }
        if (profesores.get(pos).equals(director)) {
            throw new IllegalStateException("No se puede eliminar al director");
        }
        profesores.remove(pos);
    }
}

```

```

    }

    public int getNumProfesores() {
        return profesores.size();
    }

    public Profesor getProfesor(int pos) {
        if (pos < 0 || pos >= profesores.size()) {
            throw new IllegalArgumentException("Posición inválida");
        }
        return profesores.get(pos);
    }

    public void cambiarDirector(Profesor nuevoDirector) {
        if (nuevoDirector == null) {
            throw new IllegalArgumentException("El nuevo director no puede ser
nulo");
        }
        if (!profesores.contains(nuevoDirector)) {
            throw new IllegalStateException("El nuevo director debe ser un
profesor del departamento");
        }
        this.director = nuevoDirector;
    }
}

```

10. Al igual que ocurre con las excepciones en Java, que pueden encerrar causas (que son excepciones), de forma recursiva, suponen un tipo especial de composiciones, denominadas composiciones recursivas. Pon un ejemplo en Java de una **Persona**, que sea inmutable, y que tiene una madre, que es otra **Persona**. Haz un main con un ejemplo de uso con una familia de personas, desde el nieto hasta la abuela. Enumera algún otro ejemplo clásico de composiciones recursivas.

Aquí tienes un ejemplo de una clase **Persona** que es inmutable y tiene una madre, que es otra **Persona**. Además, se incluye un **main** con un ejemplo de uso con una familia de personas, desde el nieto hasta la abuela.

```

public class Persona {
    private final String nombre;
    private final Persona madre;

    public Persona(String nombre, Persona madre) {
        this.nombre = nombre;
        this.madre = madre;
    }

    public String getNombre() {
        return nombre;
    }
}

```

```

    public Persona getMadre() {
        return madre;
    }

    public static void main(String[] args) {
        Persona abuela = new Persona("Abuela", null);
        Persona madre = new Persona("Madre", abuela);
        Persona nieto = new Persona("Nieto", madre);

        System.out.println("Nombre del nieto: " + nieto.getNombre());
        System.out.println("Nombre de la madre: " + nieto.getMadre().getNombre());
        System.out.println("Nombre de la abuela: " +
nieto.getMadre().getMadre().getNombre());
    }
}

```

## 11. ¿Qué son las relaciones de composición "bidireccionales"? ¿Qué habría que hacer para implementar este tipo de relación en el ejemplo de **Profesor** y **Departamento**?

Las relaciones de composición "bidireccionales" son aquellas en las que ambas clases involucradas tienen una referencia a la otra. En el ejemplo de **Profesor** y **Departamento**, para implementar una relación de composición bidireccional, se podría agregar una referencia al **Departamento** dentro de la clase **Profesor**. Esto permitiría que cada **Profesor** conozca a qué **Departamento** pertenece, además de que el **Departamento** conozca a sus **Profesor**.

Aquí tienes un ejemplo de cómo se podría implementar esta relación bidireccional:

```

public class Profesor {
    private String nombre;
    private Departamento departamento;

    public Profesor(String nombre, Departamento departamento) {
        this.nombre = nombre;
        this.departamento = departamento;
    }

    public String getNombre() {
        return nombre;
    }

    public Departamento getDepartamento() {
        return departamento;
    }
}

public class Departamento {
    private Profesor director;
    private List<Profesor> profesores;
}

```



```
public Departamento(Profesor director) {
    if (director == null) {
        throw new IllegalArgumentException("El director no puede ser nulo");
    }
    this.director = director;
    this.profesores = new ArrayList<>();
    this.profesores.add(director);
    director.setDepartamento(this); // Establecer la referencia al
departamento en el profesor
}

public void agregarProfesor(Profesor profesor) {
    if (profesores.size() >= 50) {
        throw new IllegalStateException("No se pueden agregar más
profesores");
    }
    profesores.add(profesor);
    profesor.setDepartamento(this); // Establecer la referencia al
departamento en el profesor
}

// Resto del código...
}
```

En este ejemplo, cada vez que se crea un **Profesor** o se agrega un **Profesor** al **Departamento**, se establece la referencia al **Departamento** en el **Profesor**. Esto permite que tanto el **Departamento** como el **Profesor** tengan conocimiento mutuo, creando así una relación de composición bidireccional.